

-



- [Core Concepts](#)
- [Workflows](#)
- Event Storage

On this page

Event Storage

Was this page helpful? 

You can configure [workflows](#) to save events that they processed to the **Event Storage**. That way, when an event reaches the end of a workflow, it is saved asynchronously to the Event Storage. Pulse saves the event's original values, as they arrived to the workflow.

The events are stored in an event storage table dedicated for the workflow schema. This table consists of columns matching the [schema's fields](#). If the workflow contains plans with scheduled metrics, then the fields that were changed by the scheduled metrics are saved in the "event diffs" column. This way Pulse can rebuild the final state of the event.

The Event Storage saves any event that reaches the end of a workflow under one of the following conditions:

- Pulse generated an exception while processing the event, causing the workflow to end.

- The event was dropped from the workflow after being processed in a service. For example, the event triggered a [rule](#) configured to stop the workflow.
- The event went through all processing elements in the workflow and correctly reached the end.

The goals of the Event Storage are the following:

- **Recovery:** If the underlying Pulse application fails or crashes unexpectedly, the Event Storage enables replaying events that have already been processed and stored, to recover the previous stable state.
- **Auditing:** Keeps track of all context regarding events by storing also:
 - All the Actions the event triggered throughout the different services in the Workflow.
 - The results of processing by Scoring Services and Rules Services, including the score and weight for each service.
 - Human-readable [explanations](#) as to why a Scoring Service or a Rules Service classified the event in a certain way.
 - Auxiliary data, such as the metrics calculated for the event.
- **Reporting:** Extract knowledge from past events and generate reports on model accuracy and outcome distribution for a consistent evaluation of runtime performance.
- **Feedback Loop:** Retrain models using the Event Storage as a source of data after receiving human input. This is especially important to close the loop on human feedback, for example, when Pulse classifies a transaction as fraud but a human analyst overrides the decision based on real-world evidence.



Disabling Event Storage

If you don't enable the event storage, Pulse will never store events in this table. This means that services that need to retrieve information from the event storage in their processing phase will not work properly.

Back End implementation📄

The Event Storage can be implemented in different back ends, such as Cassandra or a relational database.

Additionally, the back-end can process data in different ways such as binary serialization and JSON representation. The following table summarizes the advantages and drawbacks of each alternative:

	Speed	Space efficiency	Human-readable	Data migration	Query complex fields
Binary Serialization	Faster	Higher	No	Harder	No
JSON	Slower	Lower	Yes	Easier	Yes ⁽¹⁾

⁽¹⁾ In databases that support JSON natively, e.g. PostgreSQL.

Event Storage API

Configuring and accessing the contents of the Event Storage can be performed programmatically via the API.